TERRAFORM PRACTICAL ASSIGNMENT

Objective

To install Terraform on a PC, execute Terraform templates, understand Terraform commands and providers, and integrate a sample Terraform template with Jenkins.

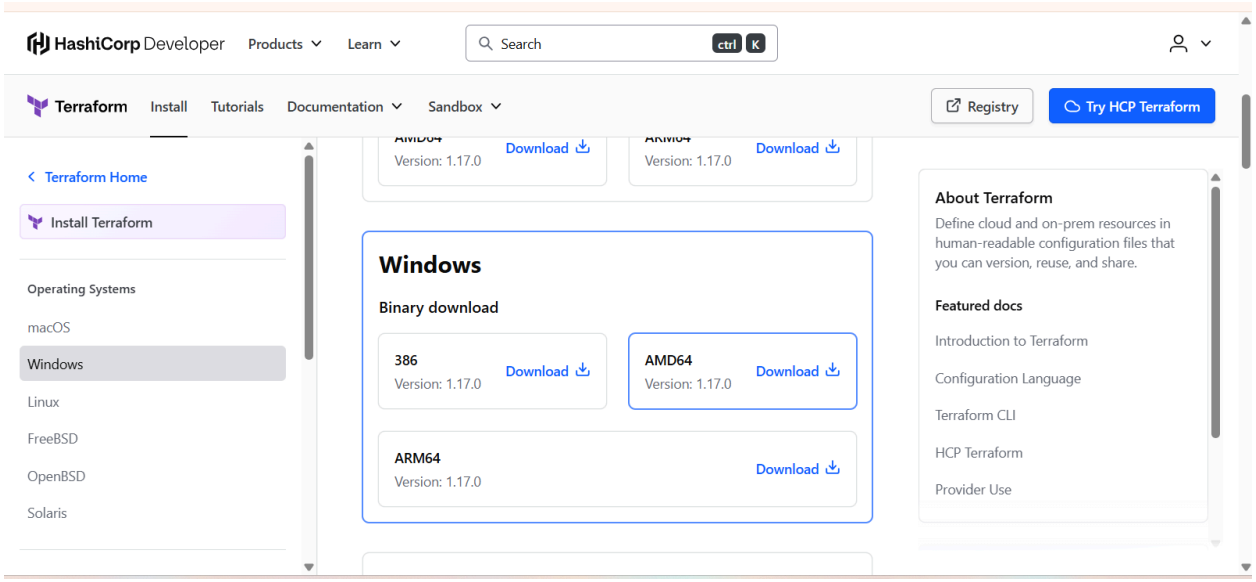
Requirements

- Windows PC
- Terraform
- Visual Studio Code
- Jenkins
- Git and GitHub
- AWS account (if executing AWS templates)

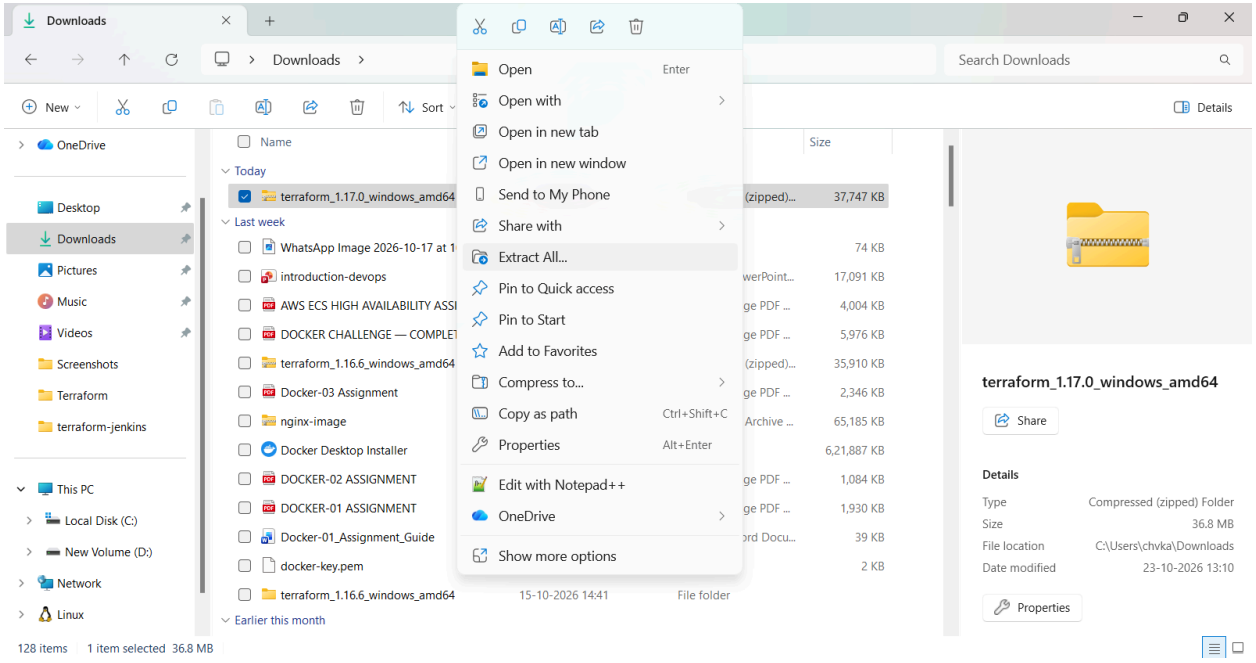
Task 1: Install Terraform on PC

Step 1: Download Terraform

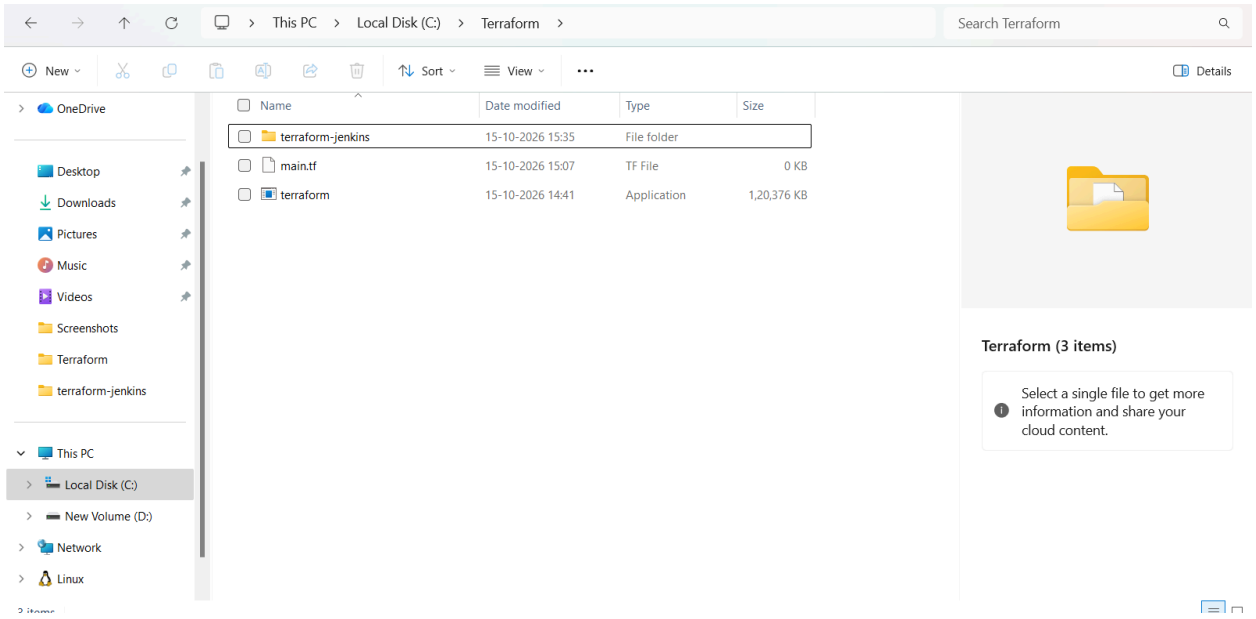
1. Open the official Terraform installation website: https://developer.hashicorp.com/terraform/install
2. Select Windows.



3. Download the appropriate ZIP file for your system.
4. Extract the ZIP file.



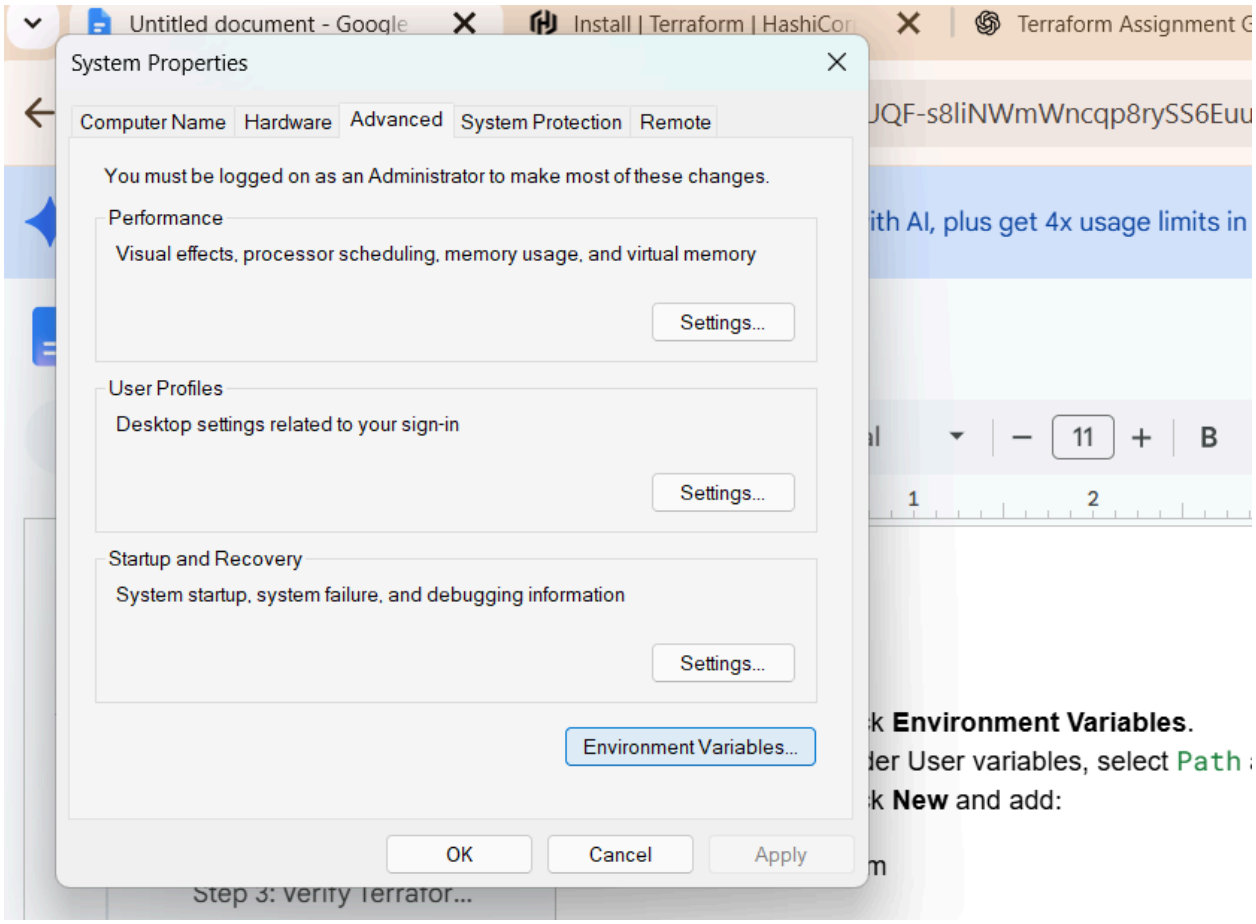
5. Create a folder named `C:\Terraform`.
6. Copy `terraform.exe` into this folder.



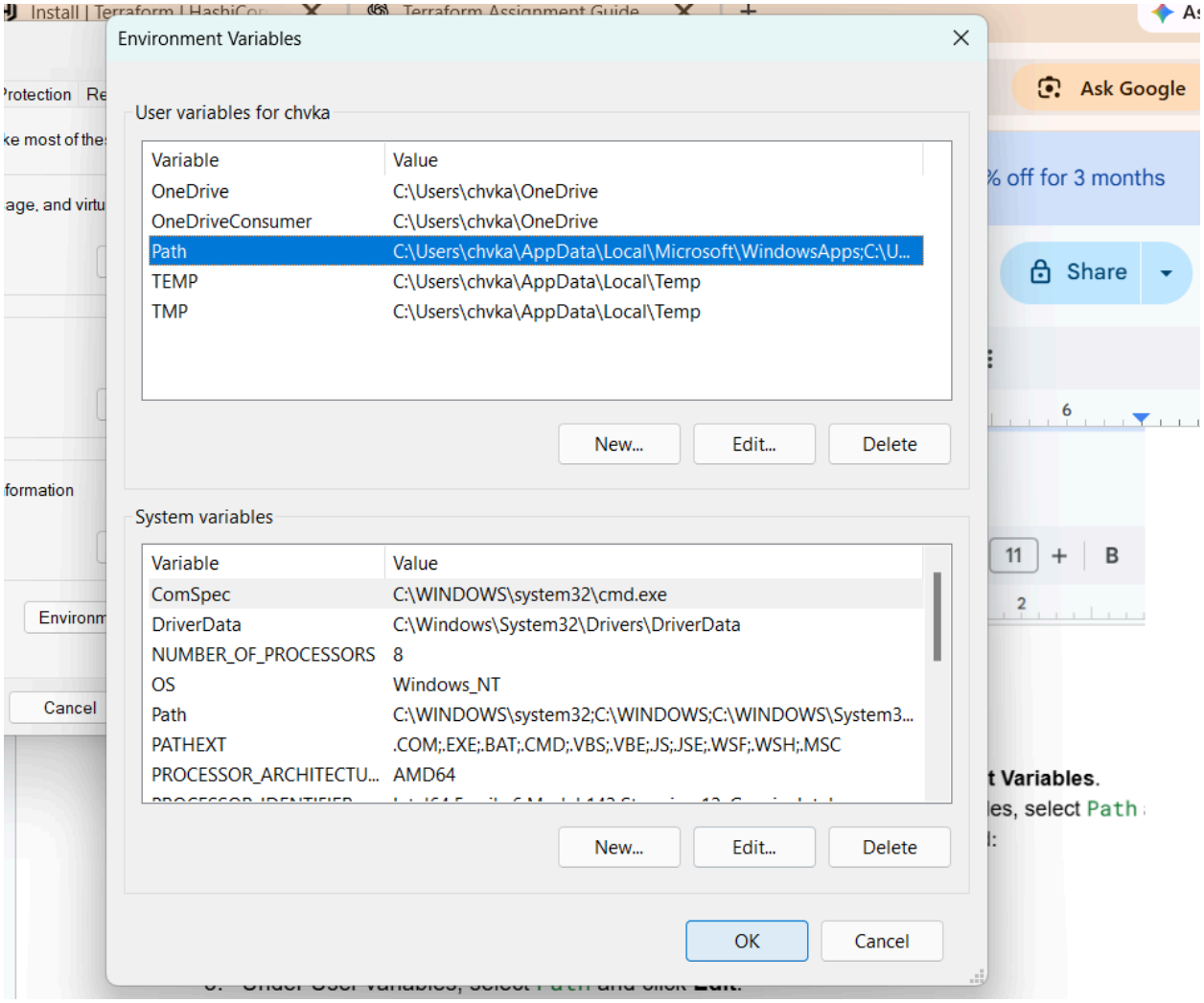
Step 2: Configure Environment Variables

1. Search for **Edit the system environment variables** in Windows.

2. Click **Environment Variables**.

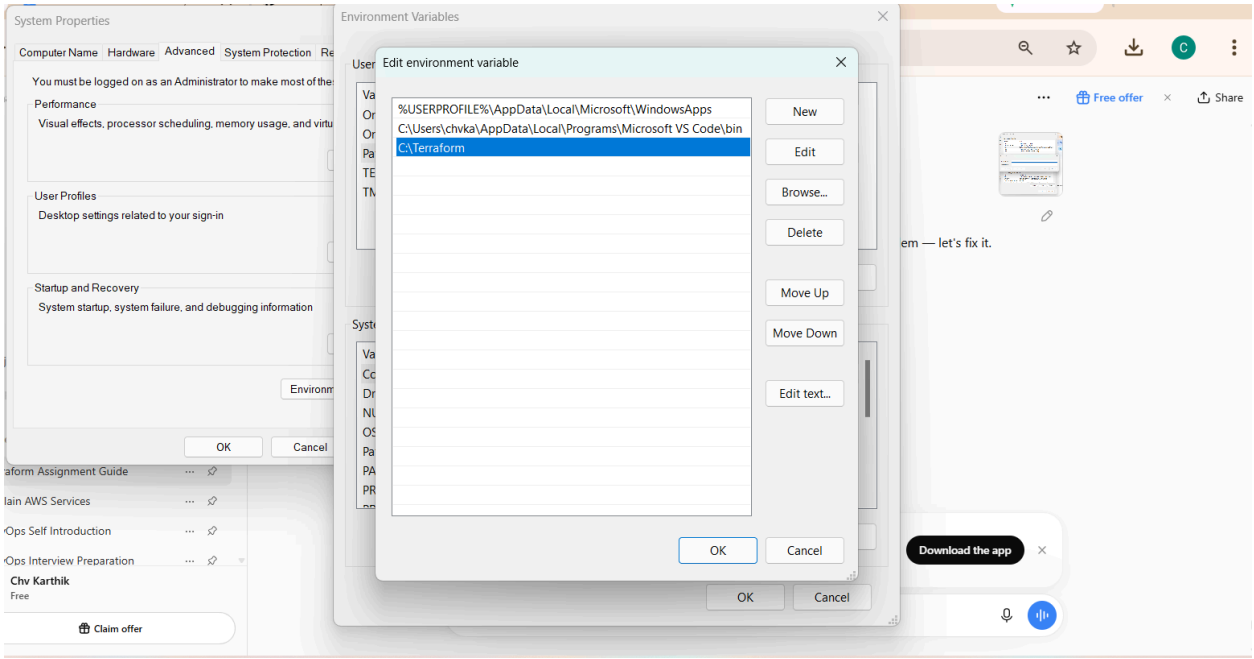


3. Under User variables, select `Path` and click **Edit**.



4. Click **New** and add:

C:\Terraform



5. Click OK and reopen PowerShell.

Step 3: Verify Terraform Installation

Execute:

terraform -version

Observation: Terraform displays its installed version, confirming that the installation was successful.

```
PS C:\Users\chvka> terraform -version
Terraform v1.16.6
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.17.0. You can update by downloading from https://developer.hashicorp.com/terraform/install
PS C:\Users\chvka>
```

Task 2: Execute Terraform Templates

Step 1: Create a Project Folder

Open PowerShell and execute:

mkdir C:\terraform-practice
cd C:\terraform-practice

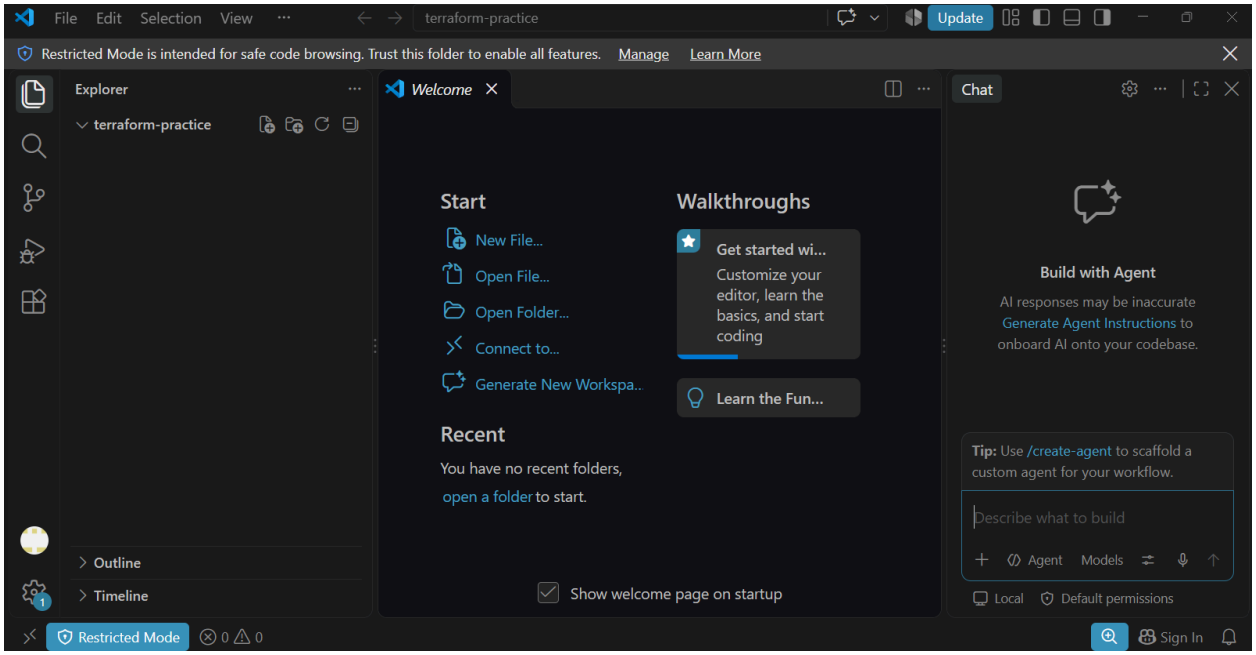
```
PS C:\Users\chvka> mkdir C:\terraform-practice


    Directory: C:\

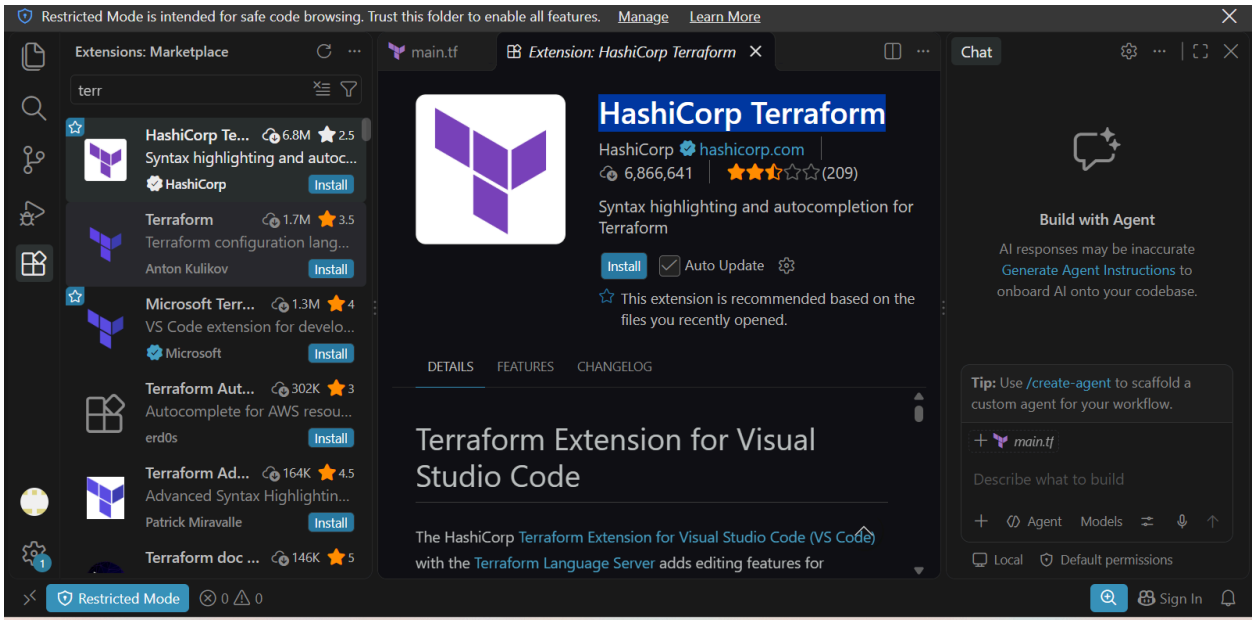

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        23-10-2026     13:19                terraform-practice


PS C:\Users\chvka> cd C:\terraform-practice
PS C:\terraform-practice>
```

Open this folder in Visual Studio Code.



Install the HashiCorp Terraform in the extension in the vs code



Step 2: Create a Terraform Configuration File

Create a file named `main.tf` and add the following configuration:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "local" {}

resource "local_file" "example" {
  filename = "${path.module}/hello.txt"
  content  = "Hello from Terraform"
}
```

After that save using ctl+s

Explanation: This configuration uses the local provider to create a text file named `hello.txt` containing the specified message.

Step 3: Initialize Terraform

terraform init

```
PS C:\terraform-practice> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.5"...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
```

Purpose: Initializes the Terraform working directory and downloads the required provider plugin.

Observation: Terraform initializes successfully.

Step 4: Validate the Configuration

terraform validate

```
PS C:\terraform-practice> terraform validate
Success! The configuration is valid.

PS C:\terraform-practice>
```

Purpose: Checks the Terraform configuration for syntax and configuration errors.

Observation: Terraform reports that the configuration is valid.

Step 5: Generate the Execution Plan

terraform plan

```
PS C:\terraform-practice> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.example will be created
  + resource "local_file" "example" {
      + content              = "Hello from Terraform"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "./hello.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\terraform-practice>
```

Purpose: Displays the changes Terraform intends to make before applying them.

Observation: The plan indicates that one local file will be created.

Step 6: Apply the Configuration

terraform apply

Review the plan and enter `yes` when prompted.

```
PS C:\terraform-practice> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.example will be created
  + resource "local_file" "example" {
      + content              = "Hello from Terraform"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "./hello.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.example: Creating...
local_file.example: Creation complete after 0s [id=e4d8040f47c75fe38d88315d46db094b4736facd]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\terraform-practice>
```

Purpose: Executes the configuration and creates the specified resource.

Step 7: Verify the Result

Get-Content .\hello.txt

Expected output:

Hello from Terraform

```
PS C:\terraform-practice> Get-Content .\hello.txt
Hello from Terraform
PS C:\terraform-practice>
```

Observation: The text file is created successfully using Terraform.

Repeat the same workflow for the additional Terraform templates provided in the instructor's video.

Task 3: Understand Terraform Commands and Provider

1. Terraform Init

Command:

terraform init

Purpose: Initializes the working directory and downloads the required providers and modules.

Usage: Execute this command when starting a Terraform project or when its dependencies change.

2. Terraform Plan

Command:

terraform plan

Purpose: Shows the proposed infrastructure changes before execution.

Usage: Execute this command to review the resources that Terraform will create, modify, or delete.

3. Terraform Apply

Command:

terraform apply

Purpose: Applies the configuration and performs the planned changes.

Usage: Execute this command after reviewing the plan and confirming that the changes are correct.

4. Terraform Provider

A provider is a plugin that allows Terraform to communicate with a platform or service such as AWS, Azure, or local files.

Example: AWS Provider

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
```

```
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

Explanation:

- `required_providers`: Specifies the required provider.
- `source`: Identifies the provider in the Terraform Registry.
- `version`: Specifies the acceptable provider version range.
- `provider "aws"`: Configures the AWS provider.
- `region`: Specifies the AWS region in which Terraform will manage resources.

Note: The AWS provider requires valid AWS credentials and appropriate IAM permissions before it can manage AWS resources.

Quick Identification Table

Topic	Command or Symbol	Main Purpose
Init	`terraform init`	Initialize the project
Plan	`terraform plan`	Preview changes
Apply	`terraform apply`	Execute changes
Provider	`provider "aws"`	Connect Terraform to AWS
Validate	`terraform validate`	Check configuration
Destroy	`terraform destroy`	Remove managed resources

Task 4: Integrate a Sample Terraform Template with Jenkins

Objective

To install Terraform on an AWS EC2 Jenkins instance, create a sample Terraform configuration, and execute Terraform commands through a Jenkins Pipeline without using Git or GitHub.

Requirements

- AWS account
- Existing Jenkins EC2 instance
- Jenkins installed and accessible through a browser
- SSH access to the EC2 instance
- Internet access for downloading Terraform providers
- Windows PC with PowerShell
- Amazon Linux EC2 instance for the steps below

Task 1: Connect to the Jenkins EC2 Instance

Step 1: Open the AWS Console

1. Log in to the AWS Management Console.
2. Search for **EC2**.
3. Click **Instances**.
4. Select your existing Jenkins EC2 instance.
5. Copy its Public IPv4 address.

Step 2: Connect through SSH

Open PowerShell on your Windows PC and run:

ssh -i "C:\path\to\your-key.pem" ec2-user@YOUR_JENKINS_PUBLIC_IP

Replace the key path and public IP with your actual values.

If prompted to confirm the host fingerprint, verify it and enter `yes`.

Purpose: Connects to the Amazon Linux EC2 instance through SSH.

Observation: The terminal connects to the instance and displays a prompt similar to:

[ec2-user@ip-172-31-20-79 ~]$

Task 2: Install Terraform on the Jenkins Instance

Important: The following commands are for Amazon Linux. Do not use Ubuntu's `apt-get` commands on this instance.

Step 1: Check the Linux version

Run:

cat /etc/os-release

Purpose: Identifies the operating system running on the EC2 instance.

Step 2: Install the required package utility

Run:

sudo dnf install -y yum-utils

If `dnf` is unavailable, try:

sudo yum install -y yum-utils

Purpose: Installs the utility required to configure the HashiCorp repository.

Step 3: Add the official HashiCorp repository

Run:

sudo yum-config-manager --add-repo
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo

Purpose: Adds the HashiCorp package repository so Terraform can be installed through the package manager.

If `yum-config-manager` is not found, check that `yum-utils` installed successfully before continuing.

Step 4: Install Terraform

Run:

sudo dnf install -y terraform

If `dnf` is unavailable, use:

sudo yum install -y terraform

Purpose: Downloads and installs Terraform from the configured repository.

Step 5: Verify the installation

Run:

terraform -version

Also run:

which terraform

```
Installed:
  terraform-1.17.0-1.x86_64

Complete!
[root@ip-172-31-20-79 ec2-user]# terraform -version
Terraform v1.17.0
on linux_amd64

[root@ip-172-31-20-79 ec2-user]# which terraform
/usr/bin/terraform
[root@ip-172-31-20-79 ec2-user]#
```

Expected result: The terminal displays the installed Terraform version and its executable path, usually `/usr/bin/terraform`.

Observation: Terraform is installed on the EC2 instance.

Reference: https://developer.hashicorp.com/terraform/install

Task 3: Verify Terraform Access from Jenkins

Terraform must be installed on the operating-system node or agent that executes the Pipeline. Installing it on the EC2 instance does not guarantee that every Jenkins agent can access it.

Step 1: Check the Jenkins execution environment

1. Open Jenkins in your browser.
2. Check which node or agent will execute the Pipeline.
3. Ensure Terraform is installed on that node.

If the Pipeline runs on the same EC2 instance, continue to the next step.

Step 2: Check the executable path

On the EC2 instance, run:

which terraform

Expected output:

/usr/bin/terraform

If Jenkins cannot find Terraform, check the PATH available to the Jenkins service or agent.

Observation: Terraform is available on the machine where the Pipeline will run.

Task 4: Create a Sample Terraform Pipeline

Step 1: Open Jenkins

Open a browser and enter:

http://YOUR_JENKINS_PUBLIC_IP:8080

Replace `YOUR_JENKINS_PUBLIC_IP` with the actual public IP address of your Jenkins instance.

Log in using your Jenkins credentials.

Observation: The Jenkins dashboard opens.

Step 2: Create a Pipeline job

1. Click **New Item**.
2. Enter the job name:
   `terraform-sample-pipeline`
3. Select **Pipeline**.
4. Click **OK**.

Step 3: Configure the Pipeline

1. Scroll down to the **Pipeline** section.
2. Set **Definition** to `Pipeline script`.
3. Do not select `Pipeline script from SCM`.
4. Paste the following script into the Script box.

```
pipeline {
    agent any

    stages {
        stage('Terraform Version') {
            steps {
                sh 'terraform -version'
            }
        }

        stage('Create Terraform File') {
            steps {
                writeFile file: 'main.tf', text: '''
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "local" {}

resource "local_file" "jenkins_example" {
  filename = "${path.module}/jenkins-output.txt"
  content  = "Terraform executed successfully through Jenkins"
}
'''
            }
        }

        stage('Terraform Init') {
            steps {
                sh 'terraform init -input=false'
            }
        }

        stage('Terraform Validate') {
            steps {
                sh 'terraform validate'
            }
        }
```

```
        stage('Terraform Plan') {
            steps {
                sh 'terraform plan -input=false'
            }
        }

        stage('Terraform Apply') {
            steps {
                sh 'terraform apply -auto-approve -input=false'
            }
        }

        stage('Verify Output') {
            steps {
                sh 'cat jenkins-output.txt'
            }
        }
    }
}
```

5. Click **Save**.

Explanation: Jenkins creates `main.tf` inside the job workspace and runs Terraform commands directly. No Git or GitHub repository is required.

Important: This script uses Linux shell commands and requires a Linux Jenkins agent with Terraform installed and internet access to download the provider.

Task 5: Execute the Jenkins Pipeline

Step 1: Start the build

1. Open `terraform-sample-pipeline`.
2. Click **Build Now**.
3. Wait for the build to finish.

Step 2: Check the output

1. Open the build number under **Build History**.
2. Click **Console Output**.
3. Review the output from each stage.

During validation, the output should include:

Success! The configuration is valid.

The final stage should display:

Terraform executed successfully through Jenkins

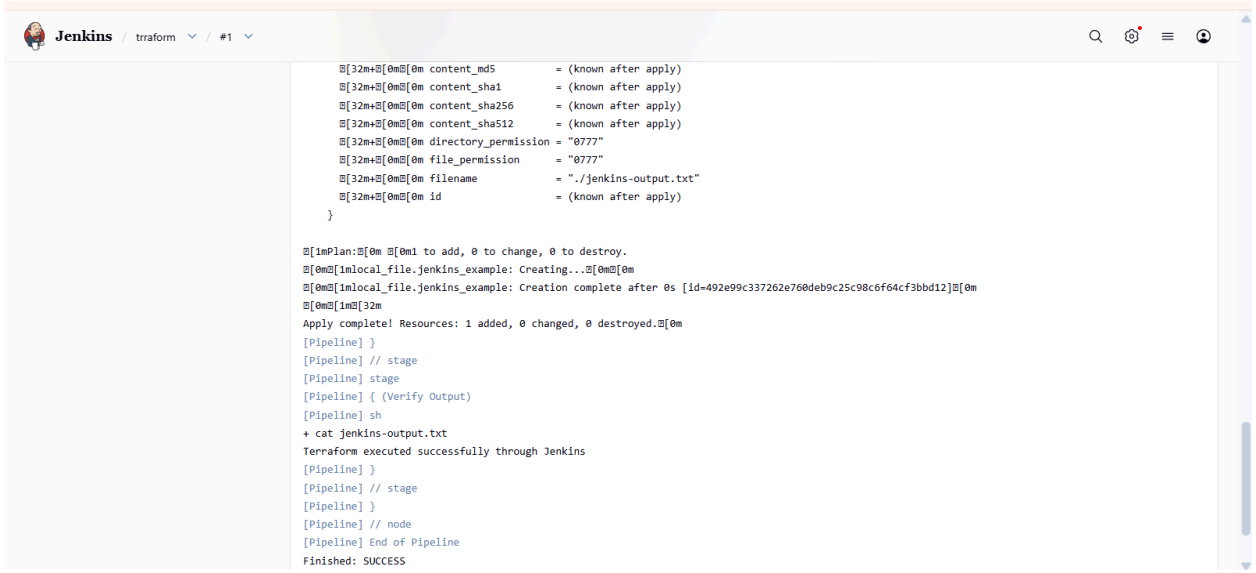
The console may also display provider installation, initialization, planning, and apply messages.

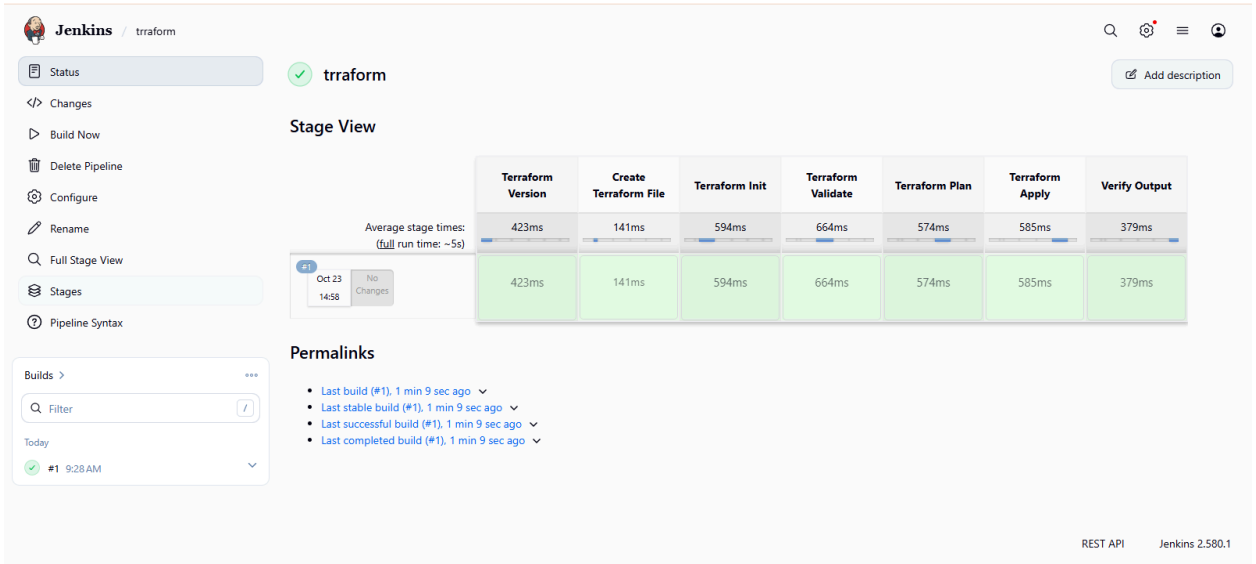
Step 3: Verify the generated file

The Pipeline creates `jenkins-output.txt` inside the job workspace.

Its contents should be:

Terraform executed successfully through Jenkins





Observation: The Jenkins Pipeline successfully runs Terraform and creates the output file.

Task 6: Understand the Terraform Commands

Command	Purpose
`terraform -version`	Displays the installed Terraform version
`terraform init`	Initializes the working directory and downloads required providers
`terraform validate`	Checks whether the configuration is valid
`terraform plan`	Previews the proposed infrastructure changes
`terraform apply`	Applies the configuration

Result

Terraform was installed on the Amazon Linux EC2 instance and verified. A sample Terraform configuration was created directly inside the Jenkins workspace and executed using a Jenkins Pipeline. Terraform initialization, validation, planning, and application were automated without using Git or GitHub.

Result

Terraform was installed and verified on the PC. Terraform templates were executed successfully, the commands `terraform init`, `terraform plan`, and `terraform apply` were studied, and the role of Terraform providers was understood. Finally, a sample Terraform configuration was integrated with Jenkins to automate its execution.

References

- Terraform Installation: https://developer.hashicorp.com/terraform/install
- Terraform CLI Documentation: https://developer.hashicorp.com/terraform/cli
- Terraform Tutorials: https://developer.hashicorp.com/terraform/tutorials
- Jenkins Pipeline Documentation: https://www.jenkins.io/doc/book/pipeline/getting-started/